

Stateless Does Not Mean Simple

Your functions are stateless. Your system is not

0
2

SERIES RECAP

The mental model we're building

01 When serverless is the wrong choice

✓ *done*

02 Stateless does not mean simple

← *You are here*

03 Why serverless is about failure domains

04 Build a minimal serverless API

hands-on build

DEFINITION

What stateless actually means

A stateless function...

- ✓ **Takes an input**
the event object — everything it needs
- ✓ **Does some work**
runs your code
- ✓ **Returns an output**
the response or result
- ✓ **Leaves no memory behind**
gone when execution ends

Why this is good

Scales trivally

Run 1,000 copies simultaneously

Safe to retry

Crash and restart without corrupting anything

Easy to test

No setup or teardown of hidden internal state

Predictable

Same input always produces the same output

MENTAL MODEL

Each invocation is isolated

Invocation A



Invocation B



Invocation C



THE MISCONCEPTION

The trap beginners fall into

The assumption

Stateless functions



Simple system

If each function is stateless and isolated, surely the whole system is easy to reason about?

Nope. The complexity doesn't disappear. It moves.

The reality

Stateless functions



Complex edges

Databases. Caches. Queues. Distributed consistency. Retry logic. Idempotency. That's where the complexity lives.

Simple functions + complex state management = complex system.

A REAL EXAMPLE

The e-commerce checkout

One stateless Lambda. Seven external operations. Multiple failure surfaces.

1 Read user's cart
DynamoDB read

2 Check inventory levels
DynamoDB read

3 Reserve inventory
DynamoDB conditional write

4 Create order record
DynamoDB write

5 Charge payment
Stripe API call

6 Send confirmation email
SES / queue

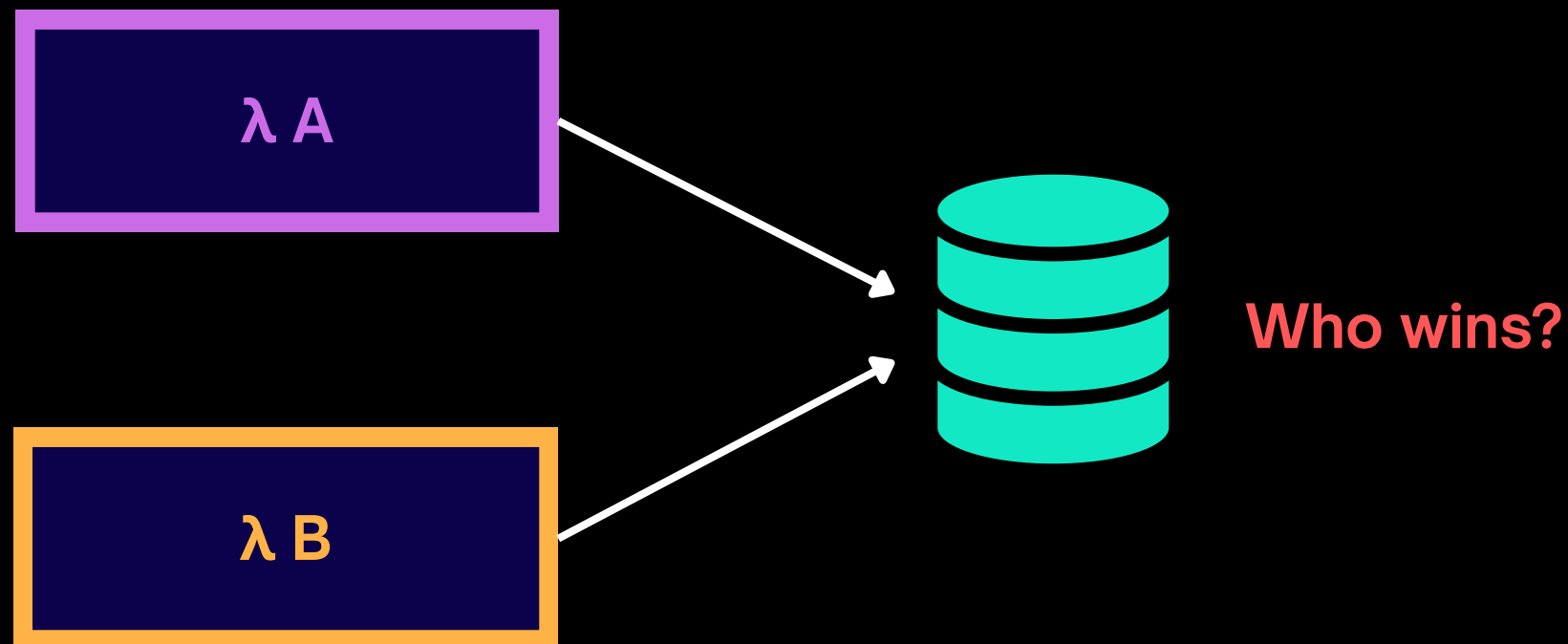
7 Update inventory as sold
DynamoDB write

If step 6 fail. Do you retry the whole function? Does the customer get charged twice?

Databases

Your primary source of truth — and the home of distributed consistency challenges.

CONCURRENT WRITE PROBLEM



What you have to think about

Transactions

All-or-nothing writes across multiple records

Optimistic locking

Detect and reject conflicting concurrent updates

Conflict resolution

When two writes race, which one wins?

Consistency model

Does every Lambda see the same data at the same time?

Caches

You add a cache to go fast. Then you have two copies of data. Now you have a new problem.



DATABASE UPDATED

DB updated: price = \$49

Cache still says: price = \$39

Next Lambda reads the cache → serves stale data → customer pays the wrong price.

Cache invalidation: famously one of the hardest problems in CS.

Queues & Idempotency

Queues let functions communicate asynchronously. But messages can arrive twice.

SQS delivery guarantees



At least once

A message may be delivered more than once



Out of order

Standard queues don't guarantee order



FIFO queues

Exactly-once, ordered (with tradeoffs on throughput)

Idempotency

Running the same function twice with the same input produces the same result, and causes no additional harm.

Idempotent: sending a duplicate email confirmation
→ 2 identical emails (annoying but harmless)

NOT idempotent: processing a payment twice → double charge

Design every queue-triggered function to be idempotent. It is the foundation of safe retries.

SUMMARY

Three forms of external state

Your function is simple. Each of these is not.



Database

Complexity: Consistency

Concurrent writes, transactions, optimistic locking, conflict resolution.

DynamoDB, RDS, Aurora



Cache

Complexity: Cache invalidation

Two copies of data. When DB changes, how does the cache know?

ElastiCache, Redis



Queue / Event stream

Complexity: Delivery guarantees

At-least-once delivery. Messages can arrive out of order. Design for idempotency.

SQS, EventBridge, Kinesis

WATCH OUT

Common gotchas

Four real bugs and misconceptions that bite serverless developers.

Warm container state leaks

Lambda reuses execution environments. Variables initialised outside your handler can persist across invocations on the same container. Never store mutable request-specific state outside the handler, write functions as if they always start cold.

FIFO queues still need idempotency

SQS FIFO deduplication only covers the 5-minute dedup window. If Lambda times out mid-processing without deleting the message, SQS redelivers it. FIFO reduces duplicate frequency. It does not eliminate the need for idempotency.

Conditional writes $\neq$ transactions

A condition on a single-item write is safe. To atomically update two items you need DynamoDB Transactions. They cost 2x read/write units and cap at 100 items per call.

Cache invalidation timing window

Even a well-designed write-through strategy has a gap between the DB write and the cache update. For prices, inventory counts, or financial data, that window can cause real harm. Design for it explicitly rather than assuming it's negligible.

Keep functions focused

One function, one job. Smaller functions fail in smaller, more predictable ways.

✗ Before: One fat function

checkoutHandler()

- reserve inventory
- create order
- charge payment
- send email
- update inventory

✓ After: Focused functions

λ reserveInventory()

λ createOrder()

λ processPayment()

λ sendConfirmation()

λ finalizeInventory()

Embrace eventual consistency

Not everything needs to succeed in the same Lambda invocation.

SYNCHRONOUS — FRAGILE



Analytics down → entire signup fails

ASYNCH — RESILIENT



Analytics down → retries independently. Signup succeeds.

Design for idempotency

Safe retries are the foundation of resilient serverless systems.

Idempotent: running a function twice with the same input = same result, no additional side effects.

✓ Idempotent — safe

Creating a note with a UUID:

```
noteld = randomUUID() → always unique
```

Duplicate POST → new noteld → two notes.
No double-charge equivalent.

✗ Not idempotent — dangerous

Processing a payment:

```
stripe.charge({ amount: 4900 })
```

**Duplicate message → second charge →
angry customer → chargeback.**

TAKEAWAYS

What you now understand

- Stateless $\neq$ simple: complexity lives in the state layer
- Three external state types: DB, cache, queue
- Design functions to be small, async, and idempotent